



PRM393 · Mobile Programming · Lab 03

Firestore-Powered Journal Trend Analyzer

Final project report covering OpenAlex research analytics, Firestore integration, MVVM architecture, automated Patrol testing, and AI-assisted quality review.

COURSE

PRM393 — Lab 03

PLATFORM

Flutter · Android

APPLICATION

Journal Trend Analyzer 1.0.0

VALIDATION DATE

17 July 2026

DATA SOURCE

OpenAlex API

CLOUD PLATFORM

Firestore (prm301-lab3)

Team members



Nguyễn Đức Huỳnh
SE193874



Võ Việt Minh Đức
SE193219



Nguyễn Minh Châu
SE180582



Hồng Lê Đăng Khoa
SE182425

1. Project Overview

Journal Trend Analyzer is a Flutter mobile application that helps users explore research topics and inspect publication, journal, keyword, and author trends. OpenAlex is the authoritative publication data source; Firebase provides identity, cloud storage, messaging, analytics, crash reporting, and remote configuration. Four persistent navigation destinations— **Home**, **Journals**, **Keywords**, and **Profile**— match the Lab 03 assignment specification.

4

Main tabs

7

Required events

6

Firebase services

11/11

E2E tests passed

1.1 Objectives

- Build a scalable Flutter app with clear MVVM separation.
 - Integrate OpenAlex for live research analytics.
 - Authenticate users with Google Sign-In via Firebase Auth.
- Use Analytics, Crashlytics, Remote Config, FCM and Storage.
 - Automate end-to-end validation with Patrol (11 scenarios).
 - Improve quality through SonarQube / AI-assisted review.

1.2 Implemented scope

Area	Implemented capabilities	Status
Authentication	Google Sign-In, Firebase Authentication user profile, sign-out and auth gate.	Complete
Home	Topic discovery/search, ranked topics, dashboard KPIs, yearly trend and publication paging.	Complete
Publication	Title, authors, year, journal, citations, DOI/original link and abstract decode.	Complete
Journals	Contribution ranking, citation/publication stats, detail drill-down.	Complete
Keywords	Frequency bars, emerging-keyword trends, research-frontier scatter and author ranking.	Complete
Profile	User identity, notifications, PDF export/history, Remote Config, Crashlytics and sign-out.	Complete

1.3 Technology stack

Client

- Flutter 3.41.9
- Dart 3.11.5
- Material 3 · go_router

State & data

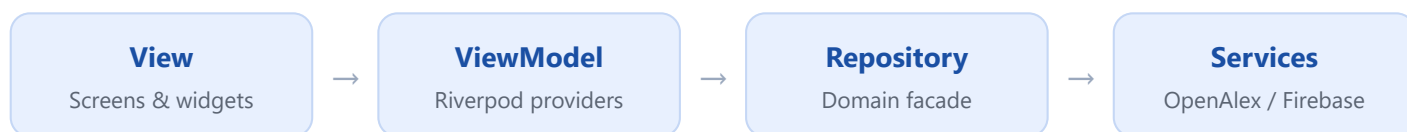
- Riverpod (ViewModels)
- Repository / UseCase
- OpenAlex via Dio

Quality & cloud

- Patrol 4.x · SonarQube
- Firebase (6 services)
- pdf → Storage

2. Architecture and MVVM Implementation

The project uses an MVVM-compatible layered design organized by feature (Clean Architecture layout). Screens remain focused on rendering and interaction. Riverpod notifiers/providers act as ViewModels, repositories expose domain operations, and services isolate OpenAlex / Firebase SDKs so UI never calls cloud APIs directly.



Layer	Main components	Responsibility
Model	Work, Author, JournalSummary, UserSettings, notification & report models	Typed application data, OpenAlex parsing, settings and report snapshots.
View	Login, Home, Journal, Keyword, Profile and detail screens	Responsive UI; loading / error / empty / data states; navigation.
ViewModel	AuthViewModel, HomeViewModel, JournalViewModel, ProfileViewModel, ...	Async orchestration, state transitions, validation and UI notifications.
Data	PublicationRepository, ApiClient, AnalyticsService, StorageService, ...	HTTP / Firebase access, PDF generation and failure translation.

Topic analysis flow

1. Normalize the query and record Analytics `search_topic` .
2. Load publications, journals and keywords asynchronously.
3. Compute top authors, influential papers, yearly trend and citation average.
4. Publish progressive state updates so the dashboard stays responsive.

Reliability decisions

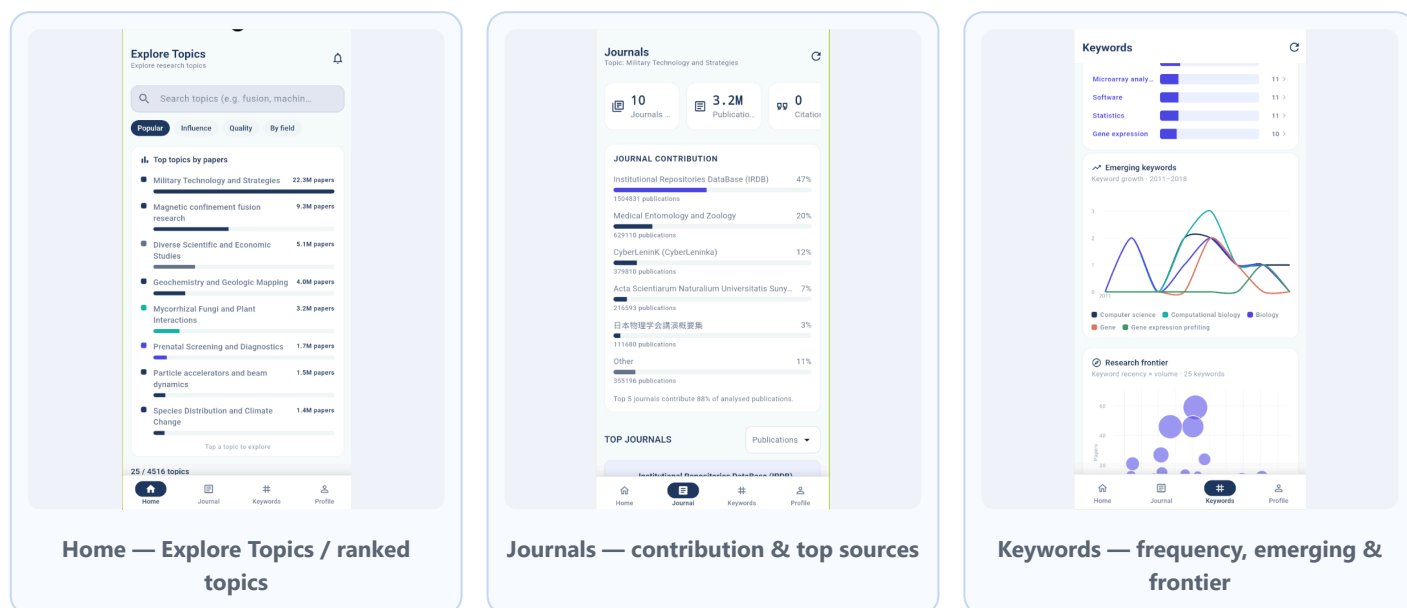
- Bounded pages and request timeouts protect the mobile client.
- Future publication years are filtered using the device year.
- Errors map to user-friendly, retryable UI states (`Either<Failure, T>`).
- PDF reports are scoped by Firebase UID under `report/{uid}/...` .

Project organization

```
lib/
├─ core/           # config, theme, router, network, shared widgets
├─ firebase/       # Auth, Analytics, Crashlytics, FCM, Remote Config, Storage
├─ features/
│   ├─ auth/       # Google Sign-In + login screen
│   ├─ home/       # Explore topics / dashboard
│   ├─ journal/    # Journal ranking + detail
│   ├─ keywords/   # Keyword analytics + charts
│   ├─ profile/    # Settings, PDF, Firebase demos
│   ├─ publication/ # Shared Work models / use cases
│   └─ shared/     # Selected topic / pending search
```

3. User Interface and Research Analytics

The UI uses Material 3 with a navy accent, shared typography/colors, rounded cards and explicit loading/error/empty states. Bottom navigation keeps four destinations alive via `StatefulShellRoute.indexedStack`. Selecting a topic drives Journals and Keywords analysis.

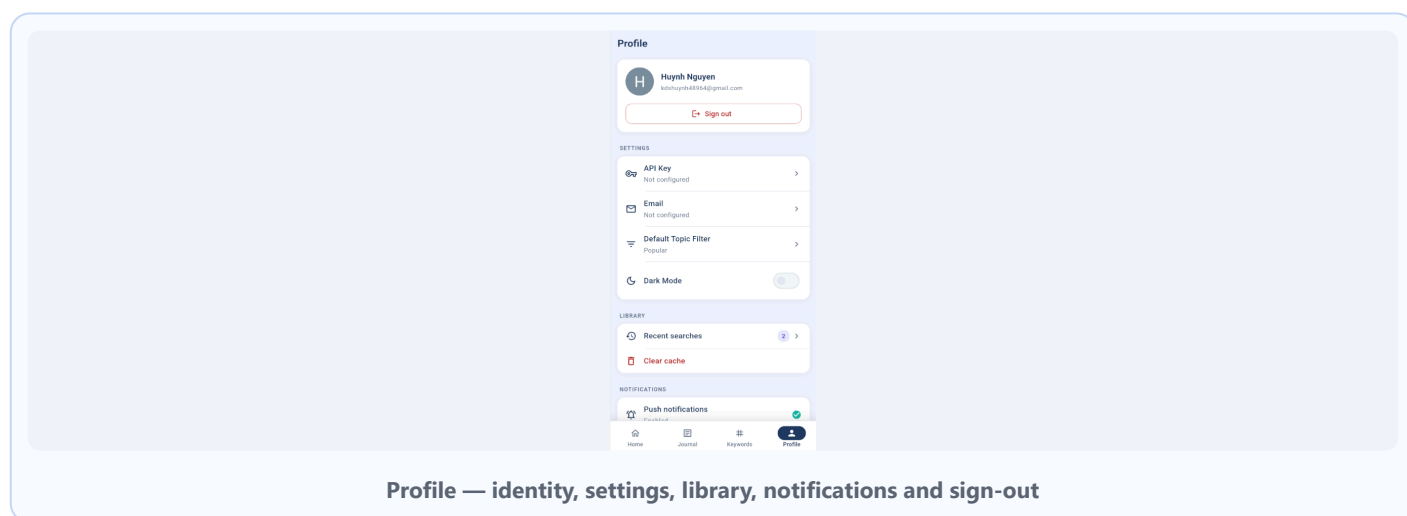


Research metrics (Home / Journals)

Home discovers topics (e.g. by papers). After a topic is selected, Journals shows KPI cards (journals count, publications, citations) and contribution bars. Publication detail exposes title, authors, year, journal, citations, DOI and decoded abstract.

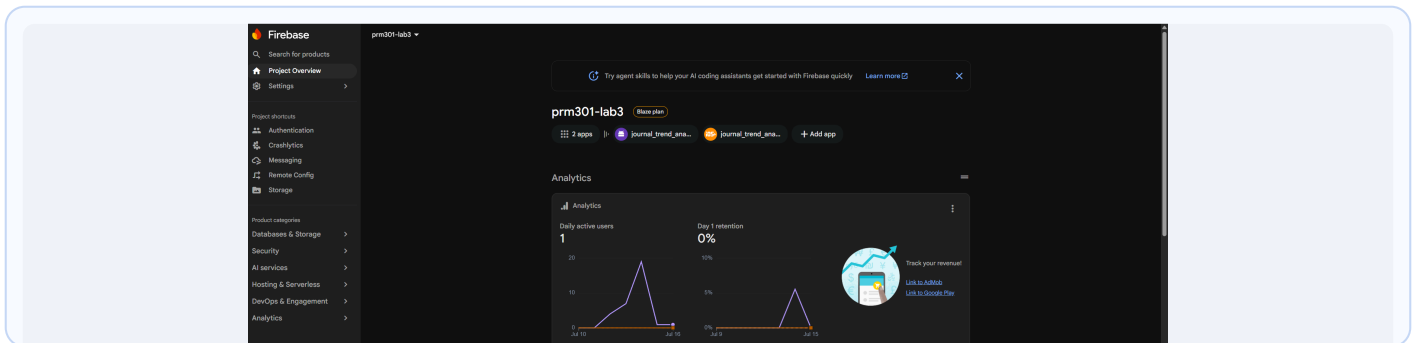
Drill-down navigation

Keyword frequency rows open detail screens with trend charts and ranked authors. Journal rows open journal detail. Profile consolidates identity, library, notifications and Firebase demonstration actions.



4. Firebase Integration and Analytics

Firebase project **prm301-lab3** (Blaze plan) hosts Android and iOS apps for *journal_trend_analyzer*. All SDK calls are wrapped in `lib/firebase/` services so ViewModels remain testable and UI stays free of direct SDK usage.



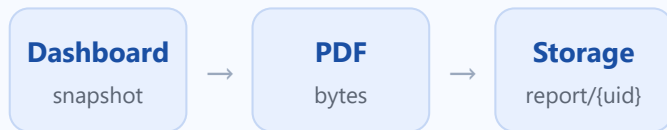
Firebase service	Application use	Evidence
Authentication	Google Sign-In, user profile, sign-out, router auth gate.	Login / Profile · Patrol TC1 & TC11
Storage	Upload / list / open / delete PDF reports under <code>report/{uid}/...</code>	Profile export · Patrol TC9
Cloud Messaging	Foreground / background / opened handling, local notifications, FCM token, notification center.	Profile notifications panel
Analytics	Seven required events with contextual parameters via <code>AnalyticsService</code>	Firebase Analytics / DebugView
Crashlytics	Global fatal handlers, handled exception and deliberate test crash on Profile.	Crashlytics console · Profile demos
Remote Config	<code>max_journals_displayed</code> , <code>max_keywords_displayed</code>	Profile demo · Patrol TC10

Required Analytics event mapping

Event	Parameters	Trigger
login	<code>login_method = google</code>	Successful Google authentication
search_topic	<code>keyword</code>	Topic analysis starts
view_publication	<code>publication_title</code> , <code>publication_year</code>	Publication detail opens
view_journal	<code>journal_name</code>	Journal detail opens
view_keyword	<code>keyword</code>	Keyword detail opens
export_pdf	<code>topic</code>	PDF generated and uploaded
logout	—	User signs out

5. Cloud Reports, Remote Config and Crash Monitoring

PDF report workflow



The generated A4 report includes the selected topic, summary metrics, yearly publication trend, top journals and top publications. Files are uploaded with metadata; Profile exposes history and downloadable URLs. Analytics logs `export_pdf` after a successful upload.

Remote Config & Crashlytics demos

Profile surfaces at least two remote parameters and two crash actions for verification:

- `max_journals_displayed`
- `max_keywords_displayed`
- **Handled exception** → non-fatal `recordError`
- **Test crash** → intentional fatal crash (verification only)

Safety note

“Handled exception” keeps the app running while still sending a diagnostic event. “Test crash” terminates the process and is used only to confirm Crashlytics ingestion after restart. FCM token and notification center are also available on Profile for Messaging verification.

Notification center

Incoming FCM messages are displayed locally (foreground) and stored in the in-app notification center so users can review research alerts without leaving the Profile tab.

Evidence: [Firebase Console shortcuts](#) (Authentication, Crashlytics, Messaging, Remote Config, Storage) and Profile UI actions. Storage path pattern: `report/{uid}/...`

6. Automated End-to-End Testing with Patrol

The final suite was executed on an Android emulator using the real app, Google account picker, OpenAlex API and Firebase project. Tests live in `patrol1_tests/` and map one-to-one to the eleven Lab 03 scenarios. Latest result: **11/11 passed**.

#	Scenario	Main verification	Result
1	Google Sign-In	Native account selection → Home	Pass
2	Topic Search	Publication / topic results displayed	Pass
3	Publication Details	Metadata, authors and abstract	Pass
4	Journals Navigation	Statistics and journal results	Pass
5	Journal Details	Works, citations and analysis	Pass
6	Keywords Navigation	Frequency statistics and rankings	Pass
7	Keyword Details	Trend and ranked authors	Pass
8	Profile Navigation	Identity, reports, notifications	Pass
9	PDF Export	Report generation + Storage upload URL	Pass
10	Remote Config	Both dynamic values displayed	Pass
11	Logout	Return to Google Sign-In screen	Pass

```
flutter analyze      → used in CI / local quality gate
patrol test -d emulator-5554 → 11 successful, 0 failed, 0 skipped
```

Test sources

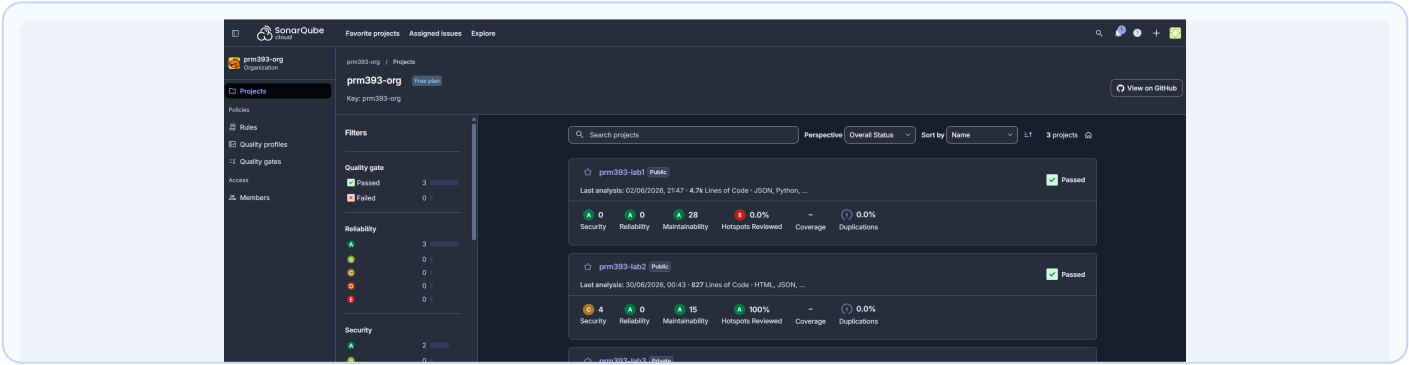
- `authentication_test.dart` (TC1, TC11)
- `publication_test.dart` (TC2, TC3)
- `journal_test.dart` (TC4, TC5)
- `keyword_test.dart` (TC6, TC7)
- `profile_test.dart` (TC8)
- `export_test.dart` (TC9)
- `remote_config_test.dart` (TC10)

Automation notes

- Selectors use stable `WidgetKeys` .
- Google Sign-In uses Patrol native automation.
- Remote Config asserts parameter names, not brittle hard-coded values.
- Stop `flutter run` before Patrol to avoid APK conflicts.

7. AI-Assisted Review, Challenges and Lessons Learned

SonarQube Cloud organization **prm393-org** was used for static analysis across the PRM393 lab series. Project **prm393-lab3** reports Quality Gate **Passed** with Reliability / Security / Maintainability grade **A** (last analysis 17/07/2026).



OK

Quality Gate

A

Reliability

A

Security

A

Maintainability

Representative findings addressed during development

Finding	Impact	Resolution	Status
Import directives inconsistently ordered	Reduced readability / analyzer consistency	Reordered imports in affected screens/ providers	Fixed
Mutable list used where a const list was enough	Unnecessary runtime allocation	Converted decorative child lists to <code>const</code>	Fixed
Remote Config E2E asserted hard-coded defaults	False failures when cloud values change	Assert rendered parameter names instead of fixed numbers	Fixed
Combined Patrol scenarios collapsed reported cases	Evidence did not map 1:1 to assignment	Split into eleven independently reported tests	Fixed

Challenges encountered

- OpenAlex rate limits and variable latency required bounded pages, timeouts and retries.
- Native Google Sign-In / permission dialogs needed Patrol native automation.
- Remote Config production values differ from local defaults — tests must assert behavior.
- Crashlytics fatal events upload after process restart.

Lessons learned

- Layered ViewModel/MVVM code is easier to test and maintain.
- Cloud evidence must be verified in Console reports, not only DebugView.
- E2E tests should map directly to assessment scenarios.
- Dynamic services need robust assertions and user-friendly fallback UI.

8. Conclusion

The **Journal Trend Analyzer** satisfies the Lab 03 functional, Firebase, architecture and testing requirements. The team delivered a production-style Flutter client with OpenAlex analytics, six Firebase services, eleven passing Patrol E2E scenarios, and a SonarQube Quality Gate Pass (grades A) for `prm393-1ab3` .

The application demonstrates maintainable MVVM layering (Riverpod ViewModels + repositories), evidence-backed cloud integration, and an automated validation path suitable for academic assessment and further product iteration.

Team contribution summary

Member	Student ID	Primary focus areas
Nguyễn Đức Huỳnh	SE193874	Architecture, Firebase integration, report & quality evidence
Võ Việt Minh Đức	SE193219	Home / Journals UI & OpenAlex analytics flows
Nguyễn Minh Châu	SE180582	Keywords analytics, charts and detail screens
Hồng Lê Đăng Khoa	SE182425	Profile, PDF export, Patrol E2E scenarios

Deliverables checklist

- Source code (Flutter + Firebase config)
 - Patrol scripts — 11 scenarios
 - This project report (HTML → PDF)
- UI screenshots (Home, Journals, Keywords, Profile)
 - Firebase Console evidence
 - SonarQube Quality Gate evidence

How to export: open this HTML in Chrome/Edge → click **Export PDF** (or Ctrl+P) → Destination: Save as PDF → disable Headers and footers → Save.